

server/internal/core 分层、保留与迁移规范

适用目录：server/internal/core/

适用阶段：开发阶段 / 产品化内测候选 / 客户试用候选前 hardening

目标：明确 Product Core 分层是否需要保留、何时迁移、哪些规则适合迁入、如何避免和现有

biz / data 主路径重复。

核心原则：core 只能承载稳定、纯粹、可复用、无 IO 的产品规则；不能成为第二套 biz，不能成为第二套 data，也不能成为第二套路由入口。

1. 总结结论

server/internal/core 可以保留，但必须严格限制边界。

推荐定位：

core = Product Core Domain Rules

也就是：

纯领域规则
状态机
值对象
业务不变量
纯计算器
领域错误
领域事件定义
与数据库无关的策略判断

不应该包含：

JSON-RPC handler
HTTP handler
Kratos service
Ent query
SQL
repository 实现
数据库事务
客户导入脚本
部署逻辑
配置文件读取
权限 session 解析
页面 DTO
transport DTO

数据库 migration
客户专属逻辑

如果当前 `core` 里已经开始出现 usecase、repository、JSON-RPC、Ent、SQL、配置读取或客户导入逻辑，应停止扩展，并逐步迁回正确位置。

2. 是否需要保留 server/internal/core

2.1 建议保留的原因

当前项目已经有比较复杂的 ERP 领域：

销售订单
采购订单
采购收货
质检
库存流水
库存余额
库存预留
生产事实
外协事实
出货
财务业务事实
BOM
SKU
客户配置
导入流程

这些领域中有很多稳定规则会被多个 usecase、API、导入工具、报表、前端页面共同依赖。

例如：

销售订单状态如何流转
采购订单如何根据收货数量更新状态
库存可用量如何计算
库存预留是否允许创建
出货是否允许发货
取消已发货出货单如何判断
BOM 是否允许激活
同一个 SKU 是否允许多个 ACTIVE BOM
财务事实是否已结清
单位换算如何计算
数量和金额是否合法

这些规则如果散落在 `biz`、`data`、JSON-RPC、导入脚本中，会导致：

规则重复
不同入口行为不一致
状态机分裂
测试困难
后续重构风险高
API、导入、页面看到不同结果

所以 `core` 有保留价值。

2.2 建议保留的条件

满足以下条件时，`core` 值得保留：

1. 规则被两个以上 usecase 复用。
2. 规则是产品核心，不是客户专属配置。
3. 规则可以用纯输入输出测试。
4. 规则不需要数据库、网络、文件、配置读取。
5. 规则会影响库存、出货、采购、财务、BOM 等关键事实。
6. 状态机复杂，散落在 usecase 中容易出错。
7. 未来可能被 JSON-RPC、REST、gRPC、CLI、导入工具、报表共同调用。

2.3 不建议保留的情况

如果 `core` 只是下面这些东西的另一个名字，就不建议保留：

第二套 biz
第二套 repository
第二套 JSON-RPC
第二套 DTO
第二套 schema
第二套导入服务
第二套 workflow
第二套客户配置系统

出现以下情况时，必须立刻停下来 Review：

biz 里有 CreateShipment, core 里也有 CreateShipment。
data 里有 InventoryRepo, core 里也有 InventoryRepo 实现。
JSON-RPC handler 同时调用 biz 和 core, 绕过统一 usecase。
core 里 import ent。
core 里 import database/sql。
core 里读取 os.Getenv。
core 里读取 config yaml。

core 里打开数据库事务。
core 里解析 JWT。
core 里检查 HTTP header。
core 里发短信、发邮件、写文件、调 webhook。
core 里出现 yoyoosun 专属字段映射。

这些都是 `core` 边界被破坏的信号。

3. 后端推荐分层

目标分层：

```
server/internal/server
  启动 HTTP/gRPC server
  注册路由
  组装 service

server/internal/service
  transport 层
  JSON-RPC / REST / gRPC handler
  参数解析
  鉴权
  权限检查
  调用 biz usecase
  错误映射
  response DTO

server/internal/biz
  应用业务用例
  事务编排
  调用 core 规则
  调用 repo interface
  写 audit event
  管理 idempotency
  触发 domain command/event
  调用 data repository

server/internal/core
  纯产品规则
  领域状态机
  值对象
  业务不变量
  纯计算器
  领域错误
  领域事件定义
  不接触数据库
  不接触网络
```

不接触 transport

```
server/internal/data
  repository 实现
  Ent query/mutation
  transaction
  row lock
  advisory lock
  persistence
  projection
  database-specific logic
```

4. 依赖方向规则

推荐依赖方向：

```
service -> biz -> core
biz -> repository interface
data -> repository implementation
data -> ent / sql
core -> no service / no biz / no data
```

严格禁止：

```
core -> service
core -> data
core -> ent
core -> sql
core -> JSON-RPC
core -> HTTP
core -> config loader
core -> os.Getenv
core -> scripts
core -> web
```

可以接受：

```
biz -> core
service -> biz
data -> core value type, 谨慎使用
data -> repository interface, 按项目风格决定
```

更严格的建议：

```
core 不 import server/internal/biz
core 不 import server/internal/data
core 不 import server/internal/service
core 不 import ent
core 不 import kratos transport
core 不 import database/sql
core 不 import net/http
core 不 import os.Getenv
```

5. core 应该放什么

5.1 值对象 Value Object

适合放入 `core` :

```
Quantity
Money
Unit
Percentage
DateRange
DocumentNo
IdempotencyKey
SourceRef
ActorRef
```

推荐目录：

```
server/internal/core/value/
  quantity.go
  money.go
  percentage.go
  unit.go
  source_ref.go
  idempotency_key.go
  document_no.go
```

这些对象应该承担基础校验：

```
数量不能为非法负数
金额精度固定
百分比不能小于 0
损耗率不能小于 0
source ref 必须有 source_type 和 source_id
idempotency key 不能为空
```

单据编号不能为空
日期范围开始不能晚于结束

为什么适合 `core` :

这些规则稳定。
多个领域复用。
不需要数据库。
非常适合单元测试。

5.2 状态机 Status Machine

适合放入 `core` :

```
SalesOrderStatusMachine  
PurchaseOrderStatusMachine  
PurchaseReceiptStatusMachine  
QualityInspectionStatusMachine  
InventoryReservationStatusMachine  
ShipmentStatusMachine  
ProductionTaskStatusMachine  
OutsourcingTaskStatusMachine  
FinanceFactStatusMachine  
BOMStatusMachine
```

推荐目录 :

```
server/internal/core/status/  
  sales_order.go  
  purchase_order.go  
  purchase_receipt.go  
  quality_inspection.go  
  inventory_reservation.go  
  shipment.go  
  production_task.go  
  outsourcing_task.go  
  finance_fact.go  
  bom.go
```

为什么适合 `core` :

状态机是产品核心规则。
状态机容易在多个 usecase 中重复。
状态机必须所有入口一致。

状态机不需要数据库。
非常适合表驱动测试。

示例：出货状态机

```
DRAFT -> READY
READY -> SHIPPED
DRAFT -> CANCELLED
READY -> CANCELLED
SHIPPED -> CANCELLED_AFTER_SHIPPED
SHIPPED -> CLOSED
CANCELLED_AFTER_SHIPPED -> CLOSED
```

禁止：

```
SHIPPED -> DRAFT
CANCELLED -> SHIPPED
CLOSED -> READY
CLOSED -> DRAFT
```

5.3 业务不变量 Invariant

适合放入 `core`：

销售订单必须有明细才能确认
客户禁用后不能创建销售订单
SKU 禁用后不能下单
采购订单 APPROVED 后才能收货
采购收货数量不能超过未收数量
BOM item qty 必须大于 0
loss_rate 必须大于等于 0
同一 SKU 只能有一个 ACTIVE BOM
ACTIVE BOM 不能直接修改 item
库存出库不能导致负库存
库存预留不能超过可用库存
重复出货不能重复扣库存
重复取消发货不能重复回滚库存
财务事实同一来源不能重复生成

推荐目录：

```
server/internal/core/rules/
  sales.go
  purchase.go
```



```
bom.go
inventory.go
shipment.go
finance.go
production.go
outsourcing.go
```

注意：

core 可以判断规则是否合法。
core 不负责加载数据库中的当前数据。
core 不负责保存状态。
core 不负责开启事务。
core 不负责写 audit。

5.4 纯计算器 Calculator

适合放入 `core`：

库存可用量计算
销售订单出货状态计算
采购订单收货状态计算
生产任务完成状态计算
外协任务回货状态计算
BOM 展开需求数量计算
应收/应付已结算状态计算
单位换算
金额汇总
数量汇总

推荐目录：

```
server/internal/core/calc/
  inventory_available.go
  bom_requirement.go
  po_receipt_status.go
  sales_shipment_status.go
  production_progress.go
  outsourcing_progress.go
  finance_settlement.go
  unit_conversion.go
```

示例规则：

可用库存 = 在库数量 - 已预留数量 - 冻结数量 - 待检数量 - 不良数量

注意：

公式必须只有一处。
页面、报表、usecase 不应该各自重写公式。
如果公式受客户配置影响，由 biz 构造 policy 后传给 core。

5.5 领域错误 Domain Error

适合放入 `core`：

```
ErrInvalidStatusTransition
ErrInsufficientInventory
ErrDuplicateActiveBOM
ErrOverReceive
ErrOverShip
ErrInactiveCustomer
ErrInactiveSKU
ErrIdempotencyConflict
ErrInvalidQuantity
ErrInvalidMoney
ErrInvalidUnitConversion
ErrInvalidBOMItem
ErrFinanceFactAlreadyExists
```

推荐目录：

```
server/internal/core/errors/
errors.go
```

为什么适合 `core`：

这些错误代表领域规则失败。
biz 可以直接返回或 wrap。
service/jsonrpc 可以统一映射成 JSON-RPC error code。

例如：

```
ErrInsufficientInventory -> BUSINESS_ERROR / INSUFFICIENT_INVENTORY
ErrInvalidStatusTransition -> BUSINESS_ERROR / INVALID_STATUS_TRANSITION
ErrDuplicateActiveBOM -> BUSINESS_ERROR / DUPLICATE_ACTIVE_BOM
```

5.6 领域事件定义 Domain Event

`core` 可以放事件定义，但不能放事件发布实现。

适合定义：

```
SalesOrderConfirmed
PurchaseOrderApproved
PurchaseReceiptCreated
QualityInspectionPassed
InventoryReserved
InventoryReservationCancelled
ShipmentShipped
ShipmentCancelledAfterShipped
FinanceFactCreated
BOMActivated
```

推荐目录：

```
server/internal/core/events/
  sales.go
  purchase.go
  inventory.go
  shipment.go
  finance.go
  bom.go
```

`core` 可以定义事件结构：

```
event type
object type
object id
source ref
occurred at
actor ref
```

`core` 不应该做：

写 event table
发 Kafka
发 webhook
发短信
发邮件
调外部系统

这些应由 `biz` 或 `infrastructure` 层处理。

5.7 Policy 类型

部分规则可能受客户配置影响，例如：

是否允许超收
是否允许超发
是否允许负库存
是否允许反审核
是否允许自动归档旧 BOM
是否需要质检
是否允许重复导入

这些可以在 `core` 中定义 policy 类型，但不能由 `core` 自己读取配置。

示例：

```
PurchasePolicy:
  AllowOverReceive
  OverReceiveTolerance

InventoryPolicy:
  AllowNegativeInventory
  ReserveExpireHours

BOMPolicy:
  AutoArchivePreviousActive
```

正确方式：

`biz` 读取客户配置
`biz` 构造 policy
`core` 根据 policy 做纯判断

错误方式：

```
core 自己读取 customer.config.json
core 自己读取环境变量
core 自己访问数据库查客户配置
```

6. core 不应该放什么

6.1 不放 JSON-RPC

禁止：

```
core/jsonrpc.go
core/handler.go
core/dispatch.go
core/public_methods.go
core/methods.go
```

原因：

JSON-RPC 是 transport 层。
core 是产品规则层。
两者不能混在一起。

应该放：

```
server/internal/service/jsonrpc/
```

6.2 不放 HTTP / Kratos service

禁止：

```
core/http.go
core/server.go
core/route.go
core/middleware.go
```

应该放：

```
server/internal/server/  
server/internal/service/
```

6.3 不放 Ent / SQL / Repository 实现

禁止：

```
core/repo.go 里直接使用 ent.Client  
core/inventory_repo.go 里写 SQL  
core/transaction.go 打开事务  
core/lock.go 做 SELECT FOR UPDATE  
core/schema.go 定义 Ent schema
```

应该放：

```
server/internal/data/
```

6.4 不放应用编排 Usecase

禁止：

```
core/CreateShipment()  
core/ShipShipment()  
core/ApprovePurchaseOrder()  
core/CreatePurchaseReceipt()  
core/ApplyImport()  
core/BootstrapAdmin()  
core/CreateFinanceFact()
```

原因：

这些动作通常需要：

```
权限  
事务  
repository  
audit  
idempotency persistence  
数据库锁  
错误映射
```

配置读取
外部 IO

应该放：

```
server/internal/biz/
```

`core` 可以提供：

```
CanShipShipment(status)
ValidateShipmentQuantities(...)
CalculateShipmentInventoryImpact(...)
NextSalesOrderStatusAfterShipment(...)
ValidateFinanceFactSourceUniqueness(...)
```

但真正执行动作应该留在 `biz`。

6.5 不放客户配置读取

禁止：

```
core.LoadCustomerConfig()
core.ReadFeatureFlags()
core.ParseEnv()
core.GetCustomerPolicyFromDB()
```

正确方式：

```
service/biz 读取配置
biz 构造 policy
core 只基于 policy 做纯判断
```

6.6 不放导入脚本和客户专属映射

禁止：

```
core/import_yoyoosun.go
core/excel_parser.go
core/source_manifest_loader.go
core/yoyoosun_mapping.go
```

应该放：

```
scripts/import/  
scripts/customer/  
server/internal/service/imports/  
server/internal/biz/imports/
```

`core` 最多提供：

```
字段值校验  
数量校验  
状态映射的纯规则  
单位换算规则
```

6.7 不放部署逻辑

禁止：

```
core/deploy.go  
core/backup.go  
core/restore.go  
core/preflight_shell.go
```

应该放：

```
scripts/deploy/  
deployments/  
server/internal/conf/
```

6.8 不放前端 DTO

禁止：

```
core/page_dto.go  
core/table_column.go  
core/antd_form.go
```

前端 DTO 应该在：


```
web/src/  
server/internal/service/jsonrpc DTO, 按项目实际拆分
```

7. 什么规则适合迁入 core

7.1 适合迁入的规则判断标准

某条规则满足以下任意两个条件，就可以考虑迁入 `core`：

1. 被两个以上 usecase 使用。
2. 已经在 biz 中重复实现。
3. 是产品核心规则，不是客户专属规则。
4. 可以用纯输入输出测试。
5. 不需要数据库、网络、文件、配置读取。
6. 状态机复杂，容易出错。
7. 会影响库存、出货、采购、财务、BOM 等关键事实。

7.2 第一批适合迁入

优先迁入低风险基础规则：

```
Quantity  
Money  
Percentage  
SourceRef  
IdempotencyKey  
DomainError
```

原因：

```
改动小。  
复用高。  
无数据库依赖。  
测试简单。  
能统一错误和参数校验。
```

7.3 第二批适合迁入

迁入稳定状态机：

ShipmentStatus
InventoryReservationStatus
PurchaseOrderStatus
PurchaseReceiptStatus
BOMStatus
FinanceFactStatus
SalesOrderStatus

原因：

状态机最容易重复。
状态机错误会直接破坏业务事实。
状态机非常适合表驱动测试。

7.4 第三批适合迁入

迁入核心计算器：

CalculateAvailableInventory
NextPurchaseOrderStatusAfterReceipt
NextSalesOrderStatusAfterShipment
ExpandBOMRequirement
CalculateFinanceSettlementStatus
CalculateProductionProgress
CalculateOutsourcingProgress

原因：

这些计算会被页面、报表、usecase、导入校验复用。
如果各自实现，容易出现同一业务对象不同状态。

7.5 第四批适合迁入

迁入受 policy 影响的复杂策略：

是否允许超收
是否允许超发
是否允许负库存
是否允许反审核
是否允许自动归档旧 BOM

是否需要质检
是否允许重复导入

迁移方式：

biz 读取客户配置
biz 构造 policy
core 根据 policy 判断

8. 不适合迁入 core 的规则

以下不适合进入 `core`：

yoyoosun 专属导入字段映射
yoyoosun 专属菜单配置
私有化部署 preflight shell 脚本
数据库连接配置
JWT 生成和解析
权限 token 解析
短信验证码发送
文件上传
Excel/PDF 解析
JSON-RPC response 格式
Ant Design 页面 DTO
SQL 查询优化
Ent schema
Atlas migration
backup/restore 脚本
Docker compose 配置
Nginx 配置
客户 raw files 处理
release package 构建

这些应该分别放到：

server/internal/conf
server/internal/service/jsonrpc
server/internal/service/imports
server/internal/biz/imports
server/internal/data
scripts/import
scripts/customer
scripts/deploy

```
deployments/yoyoosun
web/src
```

9. 典型场景分层示例

9.1 出货场景

core 负责

```
判断 shipment.status 是否允许 ship。
判断 shipment item 数量是否合法。
判断是否 over ship。
判断 reservation 是否可消耗。
计算本次出货对库存的影响。
返回领域错误。
```

biz 负责

```
检查权限。
开启事务。
加载 shipment。
加载 shipment items。
加载 inventory balance。
加载 reservation。
调用 core 校验。
调用 data repo 扣库存。
写 inventory_txns。
更新 shipment.status。
更新 sales_order_item.shipped_qty。
生成 finance_fact。
写 audit_event。
处理 idempotency persistence。
```

data 负责

```
SELECT shipment。
SELECT inventory_balance FOR UPDATE。
INSERT inventory_txns。
UPDATE inventory_balances。
UPDATE shipments。
UPDATE sales_order_items。
INSERT finance_facts。
```

service/jsonrpc 负责

解析 shipment.ship params。
读取 actor。
检查 permission shipment.ship。
调用 biz.ShipShipment。
把 core/biz 错误映射为 JSON-RPC response。

9.2 采购收货场景

core 负责

校验 PO 状态是否可收货。
校验收货数量 > 0。
校验收货数量 <= 未收数量。
计算 PO 下一个状态。
判断是否需要质检的纯策略。

biz 负责

开启事务。
加载 PO 和 PO item。
加载供应商、仓库、物料。
调用 core 校验。
创建 receipt。
更新 received_qty。
更新 PO 状态。
生成质检任务或库存入库流水。
写 audit_event。

data 负责

持久化 purchase_receipts。
更新 purchase_order_items。
更新 purchase_orders。
插入 inventory_txns。
插入 quality_inspections。

9.3 BOM 激活场景

core 负责

校验 BOM item qty。
校验 loss_rate。
校验状态迁移 DRAFT -> ACTIVE。
判断同 SKU active BOM 冲突。
根据 policy 判断是否允许自动归档旧 active BOM。

biz 负责

加载当前 BOM。
加载同 SKU active BOM。
构造 BOM policy。
调用 core 校验。
激活当前 BOM。
归档旧 BOM 或拒绝激活。
写 audit_event。

data 负责

查询 active BOM。
更新 bom_headers。
更新 bom_items。

9.4 库存预留场景

core 负责

计算可用库存。
校验预留数量。
判断 reservation 状态是否允许取消。
判断 reservation 是否可被出货消耗。

biz 负责

检查权限。
开启事务。
锁定 inventory_balance。
调用 core 计算可用库存。
创建 reservation。

处理 idempotency。
写 audit_event。

data 负责

```
SELECT inventory_balance FOR UPDATE。  
INSERT stock_reservations。  
UPDATE inventory_balances.reserved_qty。  
INSERT audit_events。
```

10. 如何避免和 biz/data 主路径重复

10.1 建立唯一职责表

规则类型	唯一归属
状态迁移是否合法	core
状态变化何时执行	biz
状态写入数据库	data
权限是否允许	service 或 biz
权限配置读取	biz/config
JSON-RPC 参数解析	service
Ent 查询	data
事务开启	biz 或 data transaction helper
行锁	data
审计事件创建	biz
审计事件持久化	data
客户配置读取	biz/config
客户配置纯策略判断	core，接收 policy 参数
导入文件解析	import service/scripts
导入数据业务校验	biz + core rules
部署 preflight	conf/scripts
release package 构建	scripts/release 或 scripts/customer

10.2 禁止重复实现状态判断

如果 `core` 已有：

```
CanTransitionShipmentStatus(from, to)
```

`biz` 中不应该再手写：

```
if status == "SHIPPED" && to == "DRAFT" {  
    return error  
}
```

`biz` 应该调用：

```
corestatus.CanTransitionShipmentStatus(from, to)
```

10.3 禁止重复实现库存公式

如果 `core` 已有：

```
CalculateAvailableInventory(onHand, reserved, frozen, pendingQC, defective)
```

其他地方不应该再手写：

```
available := onHand - reserved
```

因为这会漏掉：

```
frozen  
pending_qc  
defective
```

10.4 禁止 JSON-RPC 绕过 biz 直接调用 core

错误：

```
jsonrpc handler -> core.ValidateShipment -> data.UpdateShipment
```


正确：

```
jsonrpc handler -> biz.ShipShipment -> core.ValidateShipment -> data repo
```

原因：

biz 负责事务、权限、audit、idempotency。
绕过 biz 会破坏主路径。

10.5 禁止 data 层复制 core 规则

data 可以做数据库约束和持久化保护，但不应该复制复杂业务规则。

data 可以做：

```
unique constraint  
foreign key  
check constraint  
row lock  
idempotency unique key
```

data 不应该做：

```
自己判断销售订单状态流转  
自己决定 BOM 是否能激活  
自己决定出货是否可以发货  
自己决定财务事实是否应该生成
```

11. core 目录推荐结构

建议结构：

```
server/internal/core/  
  README.md  
  
  value/  
    quantity.go  
    money.go  
    percentage.go  
    unit.go  
    source_ref.go
```

```
idempotency_key.go
document_no.go

status/
  sales_order.go
  purchase_order.go
  purchase_receipt.go
  quality_inspection.go
  inventory_reservation.go
  shipment.go
  production_task.go
  outsourcing_task.go
  finance_fact.go
  bom.go

rules/
  sales.go
  purchase.go
  bom.go
  inventory.go
  shipment.go
  finance.go
  production.go
  outsourcing.go

calc/
  inventory_available.go
  bom_requirement.go
  po_receipt_status.go
  sales_shipment_status.go
  production_progress.go
  outsourcing_progress.go
  finance_settlement.go
  unit_conversion.go

errors/
  errors.go

events/
  sales.go
  purchase.go
  inventory.go
  shipment.go
  finance.go
  bom.go

policy/
  purchase_policy.go
  inventory_policy.go
  bom_policy.go
  shipment_policy.go
```

12. 命名规范

12.1 core 函数命名推荐

推荐使用：

```
Validate...  
Can...  
Next...  
Calculate...  
Normalize...  
Expand...  
Derive...
```

示例：

```
ValidateShipmentCanShip(...)  
CanTransitionShipmentStatus(...)  
NextPurchaseOrderStatusAfterReceipt(...)  
CalculateAvailableInventory(...)  
ExpandBOMRequirement(...)  
ValidateBOMCanActivate(...)  
CalculateFinanceSettlementStatus(...)
```

12.2 core 函数命名避免

避免使用：

```
Create...  
Update...  
Delete...  
Apply...  
Handle...  
Dispatch...  
Save...  
Load...  
Fetch...  
Query...  
Insert...
```

原因：

这些通常意味着 usecase、transport 或 persistence, 不适合 core。

12.3 core 类型命名推荐

推荐：

```
ShipmentStatus  
PurchaseOrderStatus  
InventoryPolicy  
BOMActivationPolicy  
FinanceFactStatus  
Quantity  
Money  
SourceRef  
DomainEvent
```

避免：

```
ShipmentUsecase  
InventoryRepo  
JSONRPCRequest  
EntShipment  
DBShipment  
HTTPContext  
ConfigLoader
```

13. core 测试规范

13.1 core 测试必须快速

`core` 测试必须：

```
不启动数据库  
不启动 HTTP server  
不读配置文件  
不访问网络  
不依赖真实时间  
不依赖环境变量  
不依赖文件系统
```

运行命令：

```
cd server
go test ./internal/core/...
```

目标：

数秒内完成。

13.2 必须使用表驱动测试

每个状态机、规则、计算器都应该表驱动测试：

正常路径
非法输入
边界值
重复动作
禁止迁移
policy 分支

13.3 必测规则

`core` 中每条规则都必须至少有：

正常路径测试
非法输入测试
边界值测试
状态禁止迁移测试
重复动作测试
policy 分支测试，如适用

13.4 core 测试示例

示例思路：

```
TestShipmentStatusTransition
DRAFT -> READY pass
READY -> SHIPPED pass
SHIPPED -> DRAFT fail
CANCELLED -> SHIPPED fail
CLOSED -> READY fail
```

```
TestCalculateAvailableInventory
  onHand=100 reserved=20 frozen=10 pendingQC=5 defective=5 available=60
  onHand=10 reserved=20 available should be 0 or negative guarded by policy
  pendingQC 不计入可用
```

```
TestBOMActivationRules
  DRAFT -> ACTIVE pass
  ACTIVE 修改 item fail
  qty <= 0 fail
  loss_rate < 0 fail
```

14. 自动化边界测试

14.1 core import guard

建议新增：

```
scripts/qa/core-boundary.test.mjs
```

检查：

```
core 不 import internal/data
core 不 import internal/service
core 不 import ent
core 不 import database/sql
core 不 import net/http
core 不 import os.Getenv
core 不 import kratos transport
core 不读取 config file
```

允许：

```
errors
fmt
math
strings
time, 谨慎
decimal package
标准库纯函数
server/internal/core 子包之间互相引用
```

14.2 duplicate rule scan

建议新增：

```
scripts/qa/domain-rule-duplication.test.mjs
```

扫描风险：

多处硬编码 shipment 状态迁移
多处硬编码库存可用量公式
多处硬编码 PO received status
多处硬编码 BOM active rule
多处硬编码 finance paid status

初期可以 warn，不一定 hard fail。

成熟后可变为 hard fail。

15. 迁移计划

15.1 第 0 阶段：冻结边界

目标：

core 暂停新增复杂业务逻辑。
先加 README 和 import guard。
明确目录职责。
标记当前 core 中不符合边界的文件。

任务：

1. 新增 server/internal/core/README.md。
2. 新增 scripts/qa/core-boundary.test.mjs。
3. CI 加入 core-boundary test。
4. 列出 core 中不符合边界的文件。
5. 不做大规模迁移。

完成定义：

core 边界文档存在。
core import guard 通过。
违规文件有迁移清单。

15.2 第 1 阶段：迁移值对象和错误

目标：

建立低风险 core 基础。

迁移：

Quantity
Money
Percentage
SourceRef
IdempotencyKey
DomainError

验收：

```
go test ./internal/core/...  
biz 使用 core value object  
无 Ent/SQL import  
无 transport import
```

15.3 第 2 阶段：迁移状态机

目标：

状态规则唯一化。

优先：

ShipmentStatus
InventoryReservationStatus
PurchaseOrderStatus
BOMStatus


```
FinanceFactStatus  
SalesOrderStatus
```

验收：

biz 中状态迁移全部调用 core。
旧的重复 if/else 删除。
状态机测试覆盖非法迁移。
API 行为保持一致，除明确破坏兼容的重命名外。

15.4 第 3 阶段：迁移计算器

目标：

核心计算唯一化。

优先：

```
CalculateAvailableInventory  
NextPurchaseOrderStatusAfterReceipt  
NextSalesOrderStatusAfterShipment  
ExpandBOMRequirement  
CalculateFinanceSettlementStatus
```

验收：

页面/API/报表使用同一计算逻辑。
没有重复公式。
单元测试覆盖边界值。

15.5 第 4 阶段：迁移复杂策略

目标：

客户配置驱动的规则以 policy 参数形式进入 core。

示例：

```
PurchasePolicy:  
  AllowOverReceive
```

```
OverReceiveTolerance
```

```
InventoryPolicy:
```

```
    AllowNegativeInventory
```

```
    ReservationExpireHours
```

```
BOMPolicy:
```

```
    AutoArchivePreviousActive
```

验收：

core 不读取客户配置。

biz 负责构造 policy。

测试覆盖不同 policy。

16. 暂不迁移的内容

以下内容暂时不要迁入 `core`：

```
JSON-RPC dispatch
auth public methods
permission check implementation
data repository
Ent schema
Atlas migration
yoyoosun import
source manifest loader
deployment preflight shell
backup restore
customer package build
web route
frontend menu config
release evidence
```

这些应先按原路线放到：

```
service/jsonrpc
biz
data
scripts/customer
scripts/deploy
deployments/yoyoosun
web/src
```

17. Codex 任务拆分

CORE-01：建立 core 边界文档和 import guard

任务说明：

建立 core 的边界文档和自动化边界测试，不迁移业务逻辑。

Codex 提示词：

请完成 CORE-01：建立 server/internal/core 边界文档和 import guard。

要求：

1. 新增 server/internal/core/README.md, 说明 core 只能包含纯产品规则。
2. 新增 scripts/qa/core-boundary.test.mjs。
3. 扫描 server/internal/core, 禁止 import internal/data、internal/service、ent、database/sql、net/http、os.Getenv、Kratos transport。
4. 允许 core 子包之间互相引用。
5. 把测试加入 node --test scripts/**/*.test.mjs。
6. 不迁移业务逻辑，只建立边界。
7. 输出当前发现的违规 import 清单。

验收标准：

core README 存在。
core-boundary test 存在。
node --test scripts/**/*.test.mjs 可运行。
违规项被列出。

CORE-02：迁移值对象和领域错误

任务说明：

迁移最稳定、低风险的值对象和领域错误。

Codex 提示词：

请完成 CORE-02：迁移值对象和领域错误。

要求：

1. 在 server/internal/core/value 增加 Quantity、Money、Percentage、SourceRef、IdempotencyKey。

2. 在 `server/internal/core/errors` 增加领域错误。
3. 增加表驱动单元测试。
4. 替换 `biz` 中明显重复的数量、金额、幂等 `key` 校验。
5. 不要引入 `Ent`、`SQL`、`JSON-RPC`、`HTTP`、配置读取。
6. 运行 `go test ./internal/core/...` 和相关 `biz` 测试。

验收标准：

`value object` 测试通过。
`errors` 可被 `biz` 复用。
`core-boundary test` 通过。

CORE-03：迁移出货状态机

任务说明：

将出货状态迁移规则集中到 `core`。

Codex 提示词：

请完成 CORE-03：迁移出货状态机。

要求：

1. 在 `server/internal/core/status/shipment.go` 定义 `ShipmentStatus` 和状态迁移规则。
2. 覆盖 `DRAFT`、`READY`、`SHIPPED`、`CANCELLED`、`CANCELLED_AFTER_SHIPPED`、`CLOSED`。
3. 增加非法迁移测试。
4. `biz` 出货 `usecase` 调用 `core` 状态机。
5. 删除 `biz` 中重复状态判断。
6. 不改变库存扣减逻辑。
7. 运行 `go test ./internal/core/...` 和 `shipment` 相关测试。

验收标准：

出货状态机只有 `core` 一处定义。
非法迁移测试通过。
出货 `usecase` 使用 `core`。

CORE-04：迁移库存可用量计算

任务说明：

将库存可用量计算集中到 core。

Codex 提示词：

请完成 CORE-04：迁移库存可用量计算。

要求：

1. 在 `server/internal/core/calc/inventory_available.go` 实现可用库存计算。
2. 公式必须考虑 `on_hand`、`reserved`、`frozen`、`pending_qc`、`defective`。
3. 增加边界测试。
4. `biz` 库存预留和出货检查使用同一计算器。
5. 不要在 `core` 里查询数据库。
6. 运行相关测试。

验收标准：

库存公式只有一处。
库存预留和出货共用计算器。
`core` 测试通过。

CORE-05：迁移 BOM 状态机和激活规则

任务说明：

将 BOM 状态机和激活规则集中到 core。

Codex 提示词：

请完成 CORE-05：迁移 BOM 状态机和激活规则。

要求：

1. 在 `server/internal/core/status/bom.go` 定义 `DRAFT`、`ACTIVE`、`ARCHIVED`。
2. 在 `server/internal/core/rules/bom.go` 定义 `item qty`、`loss_rate`、`ACTIVE` 不可编辑、同 SKU active rule 的纯规则。
3. 同 SKU active rule 需要由 `biz` 传入已有 active BOM 信息，`core` 不查数据库。
4. `biz` BOM usecase 调用 `core`。
5. 增加测试。

验收标准：

BOM 状态机测试通过。
BOM usecase 使用 core。
core 不 import data/ent/sql。

CORE-06：迁移采购订单收货状态计算

任务说明：

将采购订单收货后的状态计算集中到 core。

Codex 提示词：

请完成 CORE-06：迁移采购订单收货状态计算。

要求：

1. 在 server/internal/core/status/purchase_order.go 定义采购订单状态。
2. 在 server/internal/core/calc/po_receipt_status.go 实现根据 ordered_qty、received_qty 计算状态。
3. 覆盖 DRAFT、APPROVED、PARTIALLY_RECEIVED、RECEIVED、CANCELLED、CLOSED。
4. 采购收货 usecase 使用 core 计算状态。
5. 增加超收和非法状态测试。

验收标准：

采购收货状态计算只有一处。
超收测试通过。
core-boundary test 通过。

18. 人工 Review 清单

每次涉及 `server/internal/core` 的 PR 必须检查：

- [] core 是否 import 了 data/service/ent/sql/http/config。
- [] core 是否出现 Create/Update/Delete/Apply/Handle 这类 usecase 方法。
- [] core 是否读环境变量。
- [] core 是否打开数据库事务。
- [] core 是否包含客户专属逻辑。
- [] core 是否包含 JSON-RPC/HTTP DTO。
- [] biz 是否仍然负责事务、audit、repo 调用。
- [] data 是否仍然只负责 persistence。
- [] service 是否仍然负责 params/auth/error mapping。

- [] 是否产生第二套业务路径。
- [] 是否删除了 biz 中重复规则。
- [] core 是否有快速单元测试。
- [] 状态机是否覆盖非法迁移。
- [] 客户配置是否作为 policy 参数传入, 而不是 core 自己读取。
- [] 是否没有绕过权限。
- [] 是否没有绕过 idempotency persistence。
- [] 是否没有绕过 audit event。

19. 完成定义

core 分层整改完成后, 应满足:

- [] server/internal/core/README.md 存在。
- [] core-boundary 自动化测试存在并进入 CI。
- [] core 不依赖 data/service/ent/sql/http/config。
- [] core 中只有纯规则、状态机、值对象、计算器、领域错误、事件定义。
- [] biz 调用 core, 但 core 不调用 biz。
- [] JSON-RPC 不直接调用 core 绕过 biz。
- [] data 不复制 core 业务规则。
- [] 关键状态机只有一处定义。
- [] 库存可用量公式只有一处定义。
- [] core 单元测试快速稳定。
- [] 客户配置通过 policy 进入 core, 不由 core 读取。

20. 最终原则

一句话:

core 是产品规则内核, 不是应用服务, 不是数据库层, 不是 JSON-RPC 层。

更具体:

能纯函数测试的稳定规则, 可以进 core。
需要权限、事务、repo、audit、配置、IO 的逻辑, 不能进 core。
biz 编排业务, core 判断规则, data 保存事实, service 暴露接口。